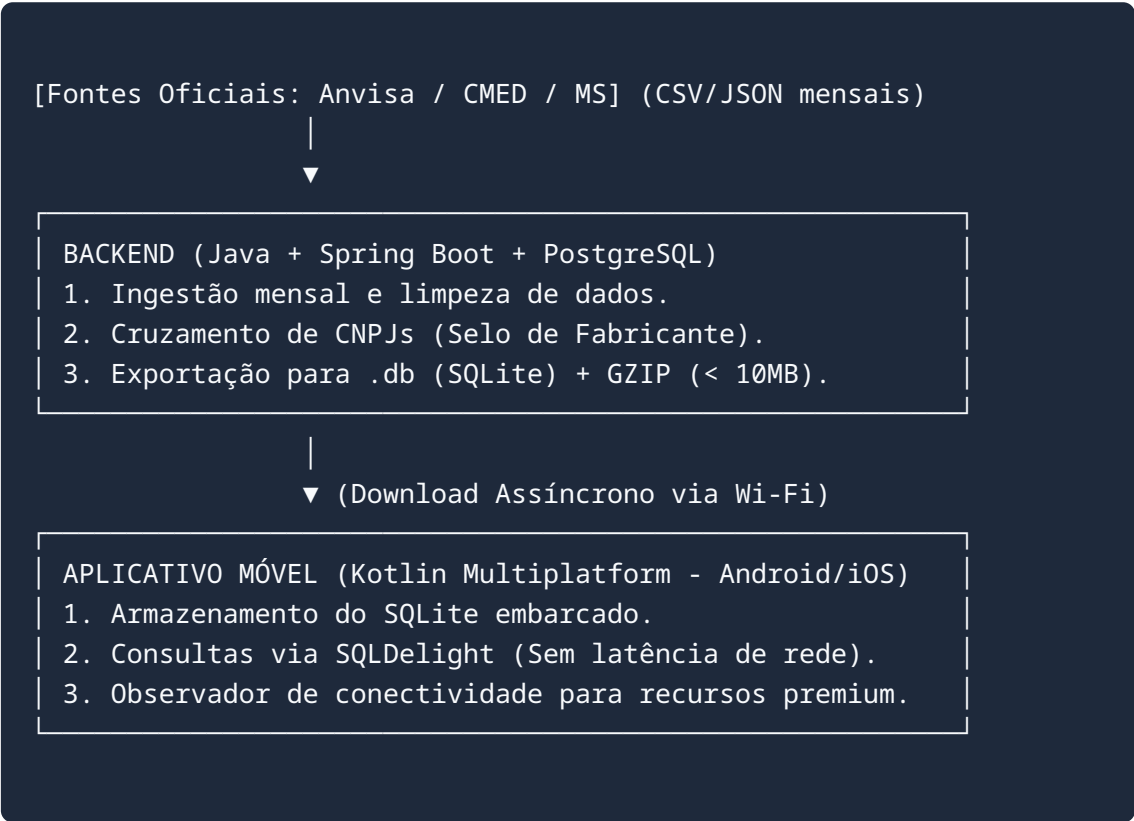


Especificação Técnica: App de Intercambialidade de Medicamentos

Objetivo do Projeto: Construir um aplicativo móvel de resposta instantânea para uso no balcão de farmácias. O app guiará a tomada de decisão do usuário (compra de genéricos e similares), operando de forma **100% offline** e garantindo a segurança clínica através de dados oficiais do governo, rejeitando técnicas frágeis como web scraping.

1. Arquitetura do Sistema

O sistema adota a arquitetura **Offline-First com Progressive Enhancement**. É dividido em um motor de consolidação de dados (Backend) e a interface de consulta instantânea (Frontend Mobile).



2. Fontes de Dados e Regras de Negócio

O aplicativo não armazena dados clínicos brutos como AUC ou Toxicidade. A segurança é garantida pelo espelhamento do registro regulatório oficial.

- **Anvisa (Dados Abertos):** Base de Medicamentos Registrados. Define o que é Referência, Genérico e Similar Intercambiável. Fornece o Princípio Ativo, Concentração, Forma Farmacêutica e o código EAN (código de barras).
- **CMED (Câmara de Regulação):** Fornece o Preço Máximo ao Consumidor (PMC) por Estado, considerando as alíquotas de ICMS locais.
- **Ministério da Saúde:** Lista de medicamentos do Programa Farmácia Popular e endereços (lat/long) das farmácias credenciadas.

3. Modelagem de Dados (PostgreSQL)

Esquema otimizado para consolidação no backend Java. Este banco gerará a versão SQLite enviada ao aplicativo.

```
CREATE TABLE fabricantes (  
    cnpj VARCHAR(14) PRIMARY KEY,  
    razao_social VARCHAR(255) NOT NULL,  
    nome_fantasia VARCHAR(255)  
);  
  
CREATE TABLE medicamentos (  
    id SERIAL PRIMARY KEY,  
    ean VARCHAR(14) UNIQUE, -- Código de barras  
    nome_comercial VARCHAR(150) NOT NULL,  
    principio_ativo TEXT NOT NULL,  
    concentracao VARCHAR(100),  
    forma_farmaceutica VARCHAR(100),  
    categoria_regulatoria VARCHAR(50),  
    tarja VARCHAR(50),  
    retencao_receita BOOLEAN DEFAULT FALSE,  
    precisa_refrigeracao BOOLEAN DEFAULT FALSE,  
    link_bula_paciente TEXT,  
    faz_parte_farmacia_popular BOOLEAN DEFAULT FALSE,  
    cnpj_fabricante VARCHAR(14) REFERENCES fabricantes(cnpj),  
    status_registro VARCHAR(20)  
);  
  
CREATE TABLE precos_cmed (  
    id SERIAL PRIMARY KEY,  
    ean VARCHAR(14) REFERENCES medicamentos(ean),  
    uf VARCHAR(2) NOT NULL,  
    pmc_zero_icms DECIMAL(10,2),  
    pmc_18_icms DECIMAL(10,2) -- Exemplo SP  
);  
  
CREATE INDEX idx_med_busca ON medicamentos (principio_ativo,  
    concentracao, forma_farmaceutica);
```

4. Especificação de Funcionalidades (Melhoria Progressiva)

Recurso	Modo Offline (Núcleo)	Modo Online (Superpoderes)
Busca por Texto ou Cód. Barras	Pesquisa instantânea no banco local. Relaciona Referência ↔ Genérico ↔ Similar.	Fallback (busca no servidor) caso o medicamento seja de um lote lançado após a última sincronização.
Selo do Fabricante	Compara localmente o CNPJ. Exibe: <i>"Fabricado pelo mesmo laboratório do medicamento de referência"</i> .	Sem alterações. Funciona perfeitamente offline.
Preço Máximo (PMC)	Exibe o preço teto local gravado na última base baixada.	Sem alterações.
Bulário Eletrônico	Desabilitado (botão visualmente inativo).	Ativado. Permite download e renderização do PDF oficial da Anvisa a partir do link armazenado no banco.
Farmácia Popular	Indica que o remédio possui desconto governamental e exibe os endereços em texto (cruzamento de GPS local).	Ativa o botão <i>"Como Chegar"</i> , enviando as coordenadas para aplicativos de rota (Google Maps/Waze).
Sincronização de Banco	Suspensa.	Background Sync: Baixa silenciosamente o novo arquivo GZIP de ~5MB do backend Java caso haja atualização mensal.

5. Plano de Implementação (Roadmap)

Fase 1: Motor de Dados (Backend Java)

1. Inicializar projeto **Spring Boot** com dependências PostgreSQL e ferramentas de processamento de CSV (ex: OpenCSV).
2. Desenvolver o *Worker* mensal: Baixar arquivos da Anvisa/CMED, limpar dados sujos, cruzar tabelas e persistir no Postgres.

3. Desenvolver o Exportador SQLite: Rotina Java que extrai os dados do Postgres para um arquivo .db, aplica compressão GZIP e disponibiliza em um endpoint REST (ex: AWS S3 ou rota direta).

Fase 2: Infraestrutura Multiplataforma (Kotlin Multiplatform)

1. Configurar ambiente KMP para centralizar a lógica de negócios e o banco de dados.
2. Implementar o **SQLDelight** para abstrair a leitura do SQLite baixado no iOS e no Android.
3. Implementar o *ConnectivityObserver* para gerenciar as transições de estado (Offline/Online) da UI.

Fase 3: Interfaces (Frontend)

1. **Android:** Criar telas utilizando *Jetpack Compose*. Integrar o ML Kit do Google para leitura de código de barras.
2. **iOS:** Criar telas utilizando *SwiftUI*, reaproveitando a mesma lógica KMP do Android.
3. Implementar os alertas visuais de retenção de receita (Tarjas) e necessidade de refrigeração (Termolábeis).